Date.prototype.getDay()

📄 Título

`Date.prototype.getDay()` en JavaScript

📝 Definición

- `getDay()` es un **método** del objeto `Date` que devuelve el **día de la semana** según la hora local, representado mediante un número entre **0 y 6**.

Valor	Día
0	Domingo
1	Lunes
2	Martes
3	Miércoles
4	Jueves
5	Viernes
6	Sábado

> **Importante:** `getDay()` devuelve el **día de la semana**, no el día del mes.

🧠 ¿Qué es?

- Es un método que permite conocer qué día de la semana corresponde a una fecha determinada.
- Devuelve un valor de tipo **number**.
- Utiliza la hora local del sistema.

🎯 ¿Para qué sirve?

Se utiliza para:

- Mostrar el nombre del día de la semana.
- Crear calendarios.
- Verificar si una fecha es fin de semana.
- Programar tareas según el día.
- Realizar validaciones relacionadas con días laborales.

⚙️ ¿Cómo funciona?

1. Se crea un objeto `Date`.
2. Se llama a `getDay()`.
3. JavaScript calcula el día de la semana correspondiente.
4. Devuelve un número entre **0 y 6**.

Ejemplo del proceso:

```
const fecha= new Date("2026-07-22");
console.log(fecha.getDay());
```

Resultado:

```
3
```

Porque el **22 de julio de 2026 es miércoles**.

📝 Sintaxis / Estructura (JavaScript)

```
fecha.getDay()
```

Parámetros

- No recibe parámetros.

Valor de retorno

- Un **number** entre **0 y 6**.
- Devuelve `NaN` si la fecha es inválida.

💡 Ejemplos

Ejemplo 1: Fecha actual

```
const hoy=new Date();
console.log(hoy.getDay());
```

Resultado (ejemplo):

```
3
```

Ejemplo 2: Fecha específica

```
const fecha=new Date("2026-12-25");
console.log(fecha.getDay());
```

Resultado:

```
5
```

Porque el **25 de diciembre de 2026 es viernes**.

Ejemplo 3: Mostrar el nombre del día

```
const dias= ["Domingo","Lunes","Martes","Miércoles","Jueves","Viernes","Sábado"
];
const fecha=new Date();
console.log(dias[fecha.getDay()]);
```

Resultado (ejemplo):

```
"Miércoles"
```

Ejemplo 4: Verificar si es fin de semana

```
const dia=new Date().getDay();
const esFinDeSemana=dia===0||dia===6;
console.log(esFinDeSemana);
```

Ejemplo 5: Fecha inválida

```
const fecha=new Date("Fecha inválida");
console.log(fecha.getDay());
```

Resultado:

```
NaN
```

⚠️ Errores comunes

❌ Confundir `getDay()` con `getDate()`

```
const fecha=new Date("2026-07-22");
console.log(fecha.getDay());
```

Resultado:

```
3
```

Mientras que:

```
console.log(fecha.getDate());
```

Resultado:

```
22
```

- `getDay()` → Día de la **semana**.
- `getDate()` → Día del **mes**.

❌ Pensar que el lunes es `0`

```
new Date().getDay();
```

En JavaScript:

```
0 → Domingo
1 → Lunes
2 → Martes
3 → Miércoles
4 → Jueves
5 → Viernes
6 → Sábado
```

❌ Llamarlo sobre una fecha inválida

```
const fecha=new Date("abc");
console.log(fecha.getDay());
```

Resultado:

```
NaN
```

💼 Casos de uso

- Mostrar el nombre del día actual.
- Determinar si una fecha es laboral.
- Verificar fines de semana.

- Programar recordatorios.
- Construir calendarios.

🧩 Relación con otros conceptos

- `getDate()` → Devuelve el día del mes (1–31).
- `getMonth()` → Devuelve el mes (0–11).
- `getFullYear()` → Devuelve el año.
- `getHours()` → Devuelve la hora.
- `Date` → Objeto que representa fechas y horas.

🧠 Analogía

Imagina que miras un calendario 📅.

`getDate()` responde:

> ¿Qué número tiene este día dentro del mes?

Por ejemplo:

```
22
```

Mientras que `getDay()` responde:

> ¿Qué día de la semana es?

Por ejemplo:

```
Miércoles
```

Es decir:

- 📅 `getDate()` → **22**
- 📅 `getDay()` → **Miércoles**

📚 Fuente

- Documentación oficial de JavaScript (MDN).
- Especificación ECMAScript.

📝 Notas Personales

- ✅ `getDay()` devuelve el **día de la semana**.
- ✅ El rango va de **0 a 6**.
- ✅ `0` representa **domingo**.
- ✅ Devuelve un valor de tipo **number**.
- ✅ Utiliza la hora local del sistema.
- ❌ No debe confundirse con `getDate()`.